

NCTU-EE ICLAB – Spring 2024

Lab09 Exercise: Coverage & Assertion

Verification: Tea House (From Lab08)

Data Preparation

1. Extract test data from TA's directory:
% tar xvf ~iclabTA01/Lab09.tar
2. The extracted LAB directory contains:
Exercise/
3. DO NOT paste the pseudo_DRAM.sv from Lab08 to the Lab09.

Description

You need to write the verification pattern for the BEV from Lab08. You need to complete following things:

1. **PATTERN.sv** (Lab09/Exercise/00_TESTBED/PATTERN.sv)
Generate pattern data.
Send pattern data to BEV.sv and bridge.sv and make sure that it will achieve coverage goals.
You also need to **check** the correctness of the output signals of the design.
2. **CHECKER.sv** (Lab09/Exercise/00_TESTBED/CHECKER.sv)
Write your cover groups and assertions here.

Specifications

Coverage:

1. Each case of **Beverage_Type** should be select at least 100 times.
2. Each case of **Bererage_Size** should be select at least 100 times.
3. Create a cross bin for the SPEC1 and SPEC2. Each combination should be selected at least 100 times. (Black Tea, Milk Tea, Extra Milk Tea, Green Tea, Green Milk Tea, Pineapple Juice, Super Pineapple Tea, Super Pineapple Tea) x (L, M, S)
4. Output signal inf.err_msg should be “No_Err”, “No_Exp”, “No_Ing“ and “Ing_OF”, each at least 20 times. (Sample the value when inf.out_valid is high)
5. Create the transitions bin for the inf.D.act[0] signal from [Make_drink:Check_Valid_Date] to [Make_drink:Check_Valid_Date]. Each transition should be hit at least 200 times. (sample the value at posedge clk iff inf.sel_action_valid)
6. Create a covergroup for material of supply action with auto_bin_max = 32, and each bin have to hit at least one time.

Notice:

1. When you send the pattern to the BEV.sv, you need to follow the specs from the Lab08. For example, all input valid signals won't overlap with each other. You can write some assertions in your CHECKER.sv to check. If you violate the specs and your assertions didn't discover but TA discover during demo, you will fail.
2. After passing the last pattern, your PATTERN should finish immediately. (Still remember not to violate any specs.)
3. During demo, TA will also use wrong design (4 cases) to test if your pattern can check the correctness of the output signals of the design. When the answer is wrong, you should stop the program immediately and display "Wrong Answer" on the terminal.
 - `"/00_run_cov"` for normal design
 - `"/00_run_cov FAIL_X"` for error answer design, e.g. `"/00_run_cov FAIL_1"`
 - 4 Error Answer Cases, from `FAIL_1` to `FAIL_4`
4. Under 01_RTL, you can use bellowing commands to check your coverage result.
 - A. `"00_run_cov"` for running your design and generate the coverage related files.
 - B. `"02_cov_detail"` for generating the detail report (01_RTL/Report/Coverage_Detail.log) of your coverage result.
 - C. `"03_cov_summary"` for generating the brief report (01_RTL/Report/Coverage_Summary.log) of your coverage result. You should ensure both of CoverGroup Average/Covered to 100% coverage.

name	CoverGroup Average	CoverGroup Covered
Checker	100.00%	100.00% (80/80)

 - D. `"imc &"` for starting Cadence IMC (Incisive Metrics Center) in gui mode. (Please refer to appendix)
 - E. `"imc -batch"` for starting Cadence IMC (Incisive Metrics Center) in batch (shell) mode.

Assertion:

1. All outputs signals (including BEV.sv and bridge.sv) should be zero after reset.
2. Latency should be less than 1000 cycles for each operation.
3. If action is completed (complete=1), err_msg should be 2'b0 (no_err).
4. Next input valid will be valid **1-4 cycles** after previous input valid fall.
5. All input valid signals won't overlap with each other.
6. Out_valid can only be high for **exactly** one cycle.
7. Next operation will be valid **1-4 cycles** after out_valid fall.

8. The input date from pattern should adhere to the real calendar. (ex: 2/29, 3/0, 4/31, 13/1 are illegal cases)
9. C_in_valid can only be high for one cycle and can't be pulled high again before C_out_valid

Notice:

1. You can't use PATTERN.sv to check the spec above.
2. Once the spec is violated, you should stop the program immediately and show the assertion message on the terminal. Your assertion warning messages should be "Assertion X is violated", where X is the number of assertions. You can directly copy the messages provided by TA in 00_TESTBED/message.txt . For normal design, please use "01_run"; For testing error design, use "01_run SPEC_X_Y", please refer to the table in grading policy.
3. The definition of cycle and latency is the same as Lab08.

Self Testing Flow

1. **./00_run_cov (For Coverage)**
TA's design + Your Checker + Your Pattern (including your dram.dat)
2. **./01_run (For Assertion)**
TA's design + Your Checker + TA's Pattern (including your dram.dat)

TA's Demo Flow

1. **(TA's DESIGN + TA's CHECKER + Your PATTERN):**
Test your pattern if it can pass TA's design and checker, and detect the error answer in TA's design (FAIL1~FAIL_4)
2. **(TA's DESIGN + TA's PATTERN+ Your CHECKER):**
Test your checker if it can pass and find every hidden error in TA's design and pattern. (SPEC_1_1 ~ SPEC_9_2)
3. **(TA's DESIGN + Your CHECKER + Your PATTERN (including your dram.dat)):**
Test your pattern and checker if their coverage rate can reach 100%, and measure their simulation time of coverage.

Note

3. Grading Policy

- **Coverage: 50%**
- 10%: Pass each item of the command "./00_run_cov" in the table below.
- 30%: 100% Coverage (requires passing the previous item).
- 10%: Simulation time of coverage (**You need to pass all specs in coverage and assertion and will get the simulation time score.**).

- **Assertion: 50%**
- You should pass all testing case to get full point for this part.

```

-----
                Congratulations!
            You have passed all patterns!
-----
Simulation complete via $finish(1) at time 3700 NS + 0
./PATTERN.v:242    $finish;
ncsim> exit

```

Command		Print From	You should Print	Points
./00_run_cov	(None)	PATTERN.sv	Congratulations	2%
	FAIL_1~FAIL_4	PATTERN.sv	Wrong Answer	8%
./01_run	(None)	PATTERN.sv	Congratulations	50%
	SPEC_1_1 ~ SPEC_1_4	CHECKER.sv	Assertion 1 is violated	
	SPEC_2_1 ~ SPEC_2_4	CHECKER.sv	Assertion 2 is violated	
	SPEC_3_1 ~ SPEC_3_2	CHECKER.sv	Assertion 3 is violated	
	SPEC_4_1 ~ SPEC_4_4	CHECKER.sv	Assertion 4 is violated	
	SPEC_5_1 ~ SPEC_5_4	CHECKER.sv	Assertion 5 is violated	
	SPEC_6_1	CHECKER.sv	Assertion 6 is violated	
	SPEC_7_1	CHECKER.sv	Assertion 7 is violated	
	SPEC_8_1 ~ SPEC_8_6	CHECKER.sv	Assertion 8 is violated	
	SPEC_9_1~ SPEC_9_2	CHECKER.sv	Assertion 9 is violated	

(The info that need to print can be copied from 00_TESTBED/message.txt)

The second demo will be 30% off.

4. 1st demo: 2024/05/06 (MON) 12:00; 2nd demo: 2024/05/08 (WED) 12:00

5. Please submit the following files by 09_submit (Will be automatically packaged by 00_tar):

- PATTERN.sv
- CHECKER.sv
- dram.dat
- Remember using **dram.dat** when you read your data in pattern. Or you will be failed at demo. You can use any software language (e.g. Python, C/Cpp) to generate.
- Usertype_BEV.sv

6. Since the purpose of this Lab is to use SystemVerilog to do verification. You should generate pattern in the PATTERN.sv directly instead of using read file method.

7. Don't use any wire/reg/submodule/parameter name called *error*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

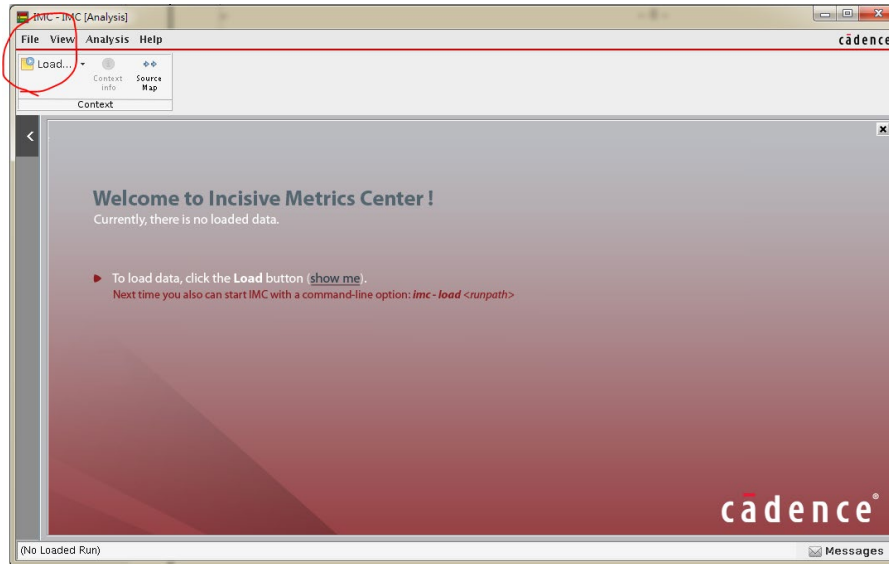
Using GUI Mode of Cadence IMC (Incisive Metrics Center)

1. % ./00_run_cov

```
4 18:03 TESTBED.SV -> ./00_TESTBED/TESTBED.S
8 18:30 cov_work
8 18:27 imc.log
8 18:30 i-run.log
```

2. % imc &

3. Load the coverage database (default path: /01_RTL/cov_work/scope/test)



Panel

List of Analytic Page

Name	Overall Average Grade	Overall Covered	Assertion Status Grade
Verification Metrics	95.83%	40 / 42 (95.24%)	100%
Types	91.67%	38 / 40 (95%)	100%
Unit	n/a	0 / 0 (n/a)	n/a
TESTBED	83.33%	36 / 38 (94.74%)	n/a
SCORE	n/a	0 / 0 (n/a)	n/a
PATTERN	n/a	0 / 0 (n/a)	n/a
Checker	100%	2 / 2 (100%)	100%
Instances	100%	2 / 2 (100%)	100%
Unit	n/a	0 / 0 (n/a)	n/a
TESTBED	100%	2 / 2 (100%)	100%

Details of Types

Name	Overall Average Grade	Overall Covered
Overall	91.67%	38 / 40 (95%)
Code	n/a	0 / 0 (n/a)
Block	n/a	0 / 0 (n/a)
Expression	n/a	0 / 0 (n/a)
Toggle	n/a	0 / 0 (n/a)
FSM	n/a	0 / 0 (n/a)
Functional	91.67%	36 / 40 (90%)
Assertion	100%	2 / 2 (100%)
CoverGroup	83.33%	36 / 38 (94.74%)

Detail:
Contribution of coverage grade
Attributes
Source code

4. Covergroup analysis

The screenshot shows the Cadence IMC - IMC [Analysis - Summary] window. The main pane displays the Verification Hierarchy, which is a tree view of the design's verification components. The 'Checker' window is open on the right, showing the 'Relative Elements' tab. The 'Checker' window displays a table of verification metrics, including 'Overall Average Grade' and 'Overall Covered' for various elements like 'check_int' and 'check_ext'.

Name	Overall Average Grade	Overall Covered
check_int	100%	108 / 108 (100%)
check_ext	100%	108 / 108 (100%)

5. Detail of items inside covergroup

The screenshot shows the Cadence IMC - IMC [Analysis - Functional] window. The main pane displays the 'CoverGroup' analysis details. The 'CoverGroup' window is open, showing a table of coverage metrics for various items. The 'latency_check' item is selected, and its details are shown in the 'Details of: latency_check' window. The 'Details of: latency_check' window displays a table of coverage metrics for various items, including 'latency_check' and 'prop_check'.

3. Detail information of your bins are shown here

1. Choose the covergroup you want to see

2. Click the item

Every coverage spec should be 100%

Name	Overall Average Grade	Overall Covered
latency_check	100%	32 / 32 (100%)
prop_check	66.67%	4 / 6 (66.67%)

6. Check your source code

The screenshot shows the Cadence JMC (Analysis - Functional) interface. The main window displays coverage analysis results for the 'prop_check' testbench. The 'COVER GROUPS' pane on the left shows the overall coverage for 'cov' (83.33%). The 'ITEMS OF: cov' pane shows the coverage for 'latency_check' (100%) and 'prop_check' (66.67%). The 'BINS OF: prop_check' pane shows the coverage for various bins (e0, e1, e2, e3, e4, e5). The 'Details of: prop_check' pane shows the source code for 'prop_check' with a red circle highlighting the 'Source' button. A red arrow points to the 'Source' button with the text 'Check your source code here'.

Overall Local Grade: 83.33% | Functional Local Grade: 83.33% | CoverGroup Local Grade: 83.33% | Assertion Local Grade: n/a

Cover Groups (83.33%) | Assertions (n/a)

Name	Overall Average Grade	Overall Covered
cov	83.33%	36 / 38 (94.74%)

Showing 1 item

ITEMS OF: cov

Name	Overall Average Grade	Overall Covered
latency_check	100%	32 / 32 (100%)
prop_check	66.67%	4 / 6 (66.67%)

Showing 2 items

BINS OF: prop_check

Name	Overall Average Grade	Overall Covered	Score
e0	0%	0 / 1 (0%)	0
e1	100%	1 / 1 (100%)	5780
e2	100%	1 / 1 (100%)	55
e3	100%	1 / 1 (100%)	30
e4	100%	1 / 1 (100%)	1255
e5	0%	0 / 1 (0%)	0

Showing 6 items

Details of: prop_check

Attributes: Source

TESTBED by

```

30 latency_check: coverpoint pf_latency(
31   option auto_bin_max = 32;
40 )
41
42 prop_check: coverpoint pf_prop(
43   bins s0 = {0..10};
44   bins s1 = {11..20};
45   bins s2 = {21..30};
46   bins s3 = {31..40};
47 )
  
```

Showing 6 items

Loaded Run: /misc/RAND2/COURSE/iclab/iclabna03/lab2_test/01_RTL/cov_work/scope/test

